

UT8 – Tarea 3

Pruebas Unitarias – Proyecto Burguer

Daniel Moreno Domínguez
Desarrollo de Interfaces – 2º DAM
Mayo 2026



1. Introducción

En esta tarea se estudian y desarrollan pruebas unitarias sobre el proyecto Burger realizado en la UT2 de Desarrollo de Interfaces. El objetivo es verificar el correcto funcionamiento de la lógica de la aplicación (modelos, cálculos de precio, gestión del carrito, validaciones, etc.) de forma automática y repetible.

Las pruebas se implementarán con JUnit 5 y seguirán el patrón Arrange–Act–Assert (AAA).

2. Descripción del Proyecto Burger

El proyecto Burger es una aplicación de escritorio desarrollada en Java con JavaFX que simula el sistema de pedidos de una hamburguesería. Las principales funcionalidades son:

- Catálogo de productos (hamburguesas, bebidas, acompañantes).
- Gestión del carrito de la compra.
- Cálculo del precio total con y sin descuentos.
- Validación de formularios (nombre del cliente, dirección de entrega, etc.).
- Persistencia básica de pedidos.

3. Plan de Pruebas Unitarias

ID	Componente	Objetivo	Resultado Esperado
PT-01	Modelo Producto	Verificar que el constructor de Producto asigna correctamente nombre, precio y categoría.	Nombre, precio y categoría iguales a los pasados al constructor.
PT-02	Modelo Producto	Comprobar que el precio nunca puede ser negativo.	Se lanza IllegalArgumentException al intentar crear un producto con precio < 0.
PT-03	Carrito – añadir ítem	Confirmar que al añadir un producto el carrito incrementa su tamaño en 1.	carrito.size() == 1 tras añadir un producto a un carrito vacío.
PT-04	Carrito – eliminar ítem	Confirmar que al eliminar un producto el carrito lo quita correctamente.	carrito.size() == 0 tras eliminar el único elemento.
PT-05	Carrito – total sin descuento	Verificar el cálculo del precio total de la cesta.	getTotalSinDescuento() == suma de (precio × cantidad) de todos los ítems.
PT-06	Descuento porcentual	Comprobar que un descuento del 10 % se aplica correctamente sobre el total.	getPrecioConDescuento(10) == totalSinDescuento * 0.90 (delta ±0.001).
PT-07	Descuento 100 %	El descuento del 100 % deja el precio a 0.	getPrecioConDescuento(100) == 0.0.
PT-08	Descuento negativo	Un descuento negativo lanza excepción.	Se lanza IllegalArgumentException.
PT-09	Validación cliente – nombre vacío	El validador rechaza un nombre en blanco.	validarNombre("") devuelve false.
PT-10	Validación cliente – nombre válido	El validador acepta un nombre correcto.	validarNombre("María García") devuelve true.
PT-11	Validación teléfono	Números de 9 dígitos son válidos; el resto no.	validarTelefono("612345678") == true; validarTelefono("123") == false.
PT-12	Pedido – estado inicial	Un pedido recién creado tiene estado PENDIENTE.	pedido.getEstado() == EstadoPedido.PENDIENTE.

PT-13	Pedido – cambio de estado	El estado se actualiza correctamente a ENTREGADO.	pedido.getEstado() == EstadoPedido.ENTREGADO tras llamar a setEstado(ENTREGADO).
PT-14	Catálogo – filtrar por categoría	El método de filtrado devuelve solo los productos de la categoría indicada.	Todos los elementos del resultado tienen la categoría BEBIDA.
PT-15	Catálogo – búsqueda vacía	Buscar una categoría inexistente devuelve lista vacía (no null).	resultado != null && resultado.isEmpty().

4. Implementación de las Pruebas Unitarias

4.1 ProductoTest (PT-01, PT-02)

Clase de prueba para el modelo Producto.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ProductoTest {

    // PT-01: Constructor asigna correctamente los atributos
    @Test
    void testConstructorAsignaAtributosCorrectamente() {
        // Arrange
        String nombre = "Hamburguesa Clásica";
        double precio = 8.50;
        String categoria = "HAMBURGUESA";

        // Act
        Producto p = new Producto(nombre, precio, categoria);

        // Assert
        assertEquals(nombre, p.getNombre(), "El nombre debe coincidir");
        assertEquals(precio, p.getPrecio(), 0.001, "El precio debe coincidir");
        assertEquals(categoria, p.getCategoria(), "La categoría debe coincidir");
    }

    // PT-02: Precio negativo lanza excepción
    @Test
    void testPrecioNegativoLanzaExcepcion() {
        // Arrange & Act & Assert
        assertThrows(IllegalArgumentException.class,
            () -> new Producto("Test", -1.0, "BEBIDA"),
            "Precio negativo debe lanzar IllegalArgumentException");
    }
}
```

4.2 CarritoTest (PT-03, PT-04, PT-05)

Clase de prueba para la gestión del carrito de la compra.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CarritoTest {

    private Carrito carrito;
    private Producto burger;

    @BeforeEach
    void setUp() {
        carrito = new Carrito();
        burger = new Producto("Hamburguesa Doble", 10.00, "HAMBURGUESA");
    }

    // PT-03: Añadir un producto incrementa el tamaño
    @Test
    void testAñadirProductoIncrementaTamano() {
        carrito.anadir(burger, 1);
        assertEquals(1, carrito.size(), "El carrito debe tener 1 elemento");
    }

    // PT-04: Eliminar el producto deja el carrito vacío
    @Test
    void testEliminarProductoDejaCarritoVacio() {
        carrito.anadir(burger, 1);
        carrito.eliminar(burger);
        assertEquals(0, carrito.size(), "El carrito debe estar vacío");
    }

    // PT-05: Total sin descuento es correcto
    @Test
    void testTotalSinDescuentoCorrecto() {
        carrito.anadir(burger, 2); // 2 × 10.00 = 20.00
        Producto bebida = new Producto("Refresco", 2.50, "BEBIDA");
        carrito.anadir(bebida, 1); // 1 × 2.50 = 2.50
        assertEquals(22.50, carrito.getTotalSinDescuento(), 0.001,
            "El total debe ser 22.50");
    }
}
```

4.3 DescuentoTest (PT-06, PT-07, PT-08)

Clase de prueba para el servicio de cálculo de descuentos.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class DescuentoTest {

    private final DescuentoService svc = new DescuentoService();

    // PT-06: Descuento del 10 % correcto
    @Test
    void testDescuentoDiezPorCiento() {
        double resultado = svc.aplicarDescuento(100.0, 10);
        assertEquals(90.0, resultado, 0.001, "100 con 10% de descuento = 90");
    }

    // PT-07: Descuento del 100 % deja precio a 0
    @Test
    void testDescuentoCienPorCienDejaCero() {
        assertEquals(0.0, svc.aplicarDescuento(50.0, 100), 0.001);
    }

    // PT-08: Descuento negativo lanza excepción
    @Test
    void testDescuentoNegativoLanzaExcepcion() {
        assertThrows(IllegalArgumentException.class,
            () -> svc.aplicarDescuento(50.0, -5));
    }
}
```

4.4 ValidadorClienteTest (PT-09, PT-10, PT-11)

Clase de prueba para las validaciones de datos del cliente.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class ValidadorClienteTest {

    private final ValidadorCliente val = new ValidadorCliente();

    // PT-09: Nombre vacío → false
    @Test
    void testNombreVacioEsInvalido() {
        assertFalse(val.validarNombre(""), "Nombre vacío debe ser inválido");
    }

    // PT-10: Nombre correcto → true
    @Test
```

```

void testNombreValidoEsAceptado() {
    assertTrue(val.validarNombre("María García"),
        "Nombre válido debe ser aceptado");
}

// PT-11: Validación de teléfono (9 dígitos)
@Test
void testValidacionTelefono() {
    assertTrue(val.validarTelefono("612345678"), "Teléfono de 9 dígitos OK");
    assertFalse(val.validarTelefono("123"), "Teléfono corto → false");
    assertFalse(val.validarTelefono("6123456789"), "Teléfono largo → false");
    assertFalse(val.validarTelefono("61234567A"), "Con letras → false");
}
}

```

4.5 PedidoTest (PT-12, PT-13)

Clase de prueba para el modelo de Pedido.

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class PedidoTest {

    // PT-12: Estado inicial es PENDIENTE
    @Test
    void testEstadoInicialEsPendiente() {
        PedidoBurger pedido = new Pedido(new Carrito(), new Cliente("Test",
"612345678"));
        assertEquals(EstadoPedido.PENDIENTE, pedido.getEstado(),
            "El estado inicial debe ser PENDIENTE");
    }

    // PT-13: Cambio de estado a ENTREGADO
    @Test
    void testCambioDeEstadoAEntregado() {
        PedidoBurger pedido = new Pedido(new Carrito(), new Cliente("Test",
"612345678"));
        pedido.setEstado(EstadoPedido.ENTREGADO);
        assertEquals(EstadoPedido.ENTREGADO, pedido.getEstado(),
            "El estado debe haber cambiado a ENTREGADO");
    }
}

```

4.6 CatalogoTest (PT-14, PT-15)

Clase de prueba para el catálogo de productos.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

class CatalogoTest {

    private Catalogo catalogo;

    @BeforeEach
    void setUp() {
        catalogo = new Catalogo();
        catalogo.agregar(new Producto("Hamburguesa Clásica", 8.5, "HAMBURGUESA"));
        catalogo.agregar(new Producto("Hamburguesa BBQ", 9.5, "HAMBURGUESA"));
        catalogo.agregar(new Producto("Refresco Cola", 2.0, "BEBIDA"));
    }

    // PT-14: Filtrar por categoría BEBIDA devuelve solo bebidas
    @Test
    void testFiltrarPorCategoriaBebida() {
        List<Producto> bebidas = catalogo.filtrarPorCategoria("BEBIDA");
        assertFalse(bebidas.isEmpty(), "Debe haber al menos una bebida");
        bebidas.forEach(p ->
            assertEquals("BEBIDA", p.getCategoria(),
                "Todos los resultados deben ser de categoría BEBIDA"));
    }

    // PT-15: Categoría inexistente devuelve lista vacía, no null
    @Test
    void testCategoriaInexistenteDevuelveListaVacía() {
        List<Producto> resultado = catalogo.filtrarPorCategoria("POSTRE");
        assertNotNull(resultado, "El resultado no debe ser null");
        assertTrue(resultado.isEmpty(), "La lista debe estar vacía");
    }
}
```

5. Resumen de Resultados

Resumen de la ejecución:

Métrica	Valor
Total de pruebas	15
Superadas 	15
Fallidas 	0
Ignoradas 	0
Tiempo de ejecución	< 1 s